Experiment No. 1: Version Control Operations using Git

Name: Afifa Syed **Roll no : 2511139** **Batch:T23**

Aim

To learn and perform basic version control operations using Git, including creating a repository, creating branches, committing changes, merging branches, and pushing changes to a remote repository.

Theory

What is Version Control?

Version Control is a system that records changes made to files over time so that specific versions can be recalled later. It allows multiple developers to work on the same project simultaneously, keeps track of changes, and helps restore previous versions if needed.

Advantages of Version Control

- Maintains a complete history of project changes.
- Enables collaboration among multiple developers.
- Prevents loss of code due to accidental deletion.
- Makes it easy to track and fix bugs.
- Supports branching and merging for parallel development.

What is Git?

Git is a distributed version control system developed by **Linus Torvalds** in 2005. It allows developers to manage source code efficiently, track modifications, and collaborate with others. Every developer has a complete copy of the repository, making Git fast and reliable.

Features of Git

- Distributed architecture.
- Fast performance.
- Branching and merging support.
- Secure version tracking using SHA hashes.
- Open-source and cross-platform.
- Supports collaboration through remote repositories like GitHub and GitLab.

Git Repository

A **repository (repo)** is a storage location where Git keeps all project files along with their version history.

There are two types:

1. **Local Repository** – Stored on the developer's computer.
2. **Remote Repository** – Hosted on platforms like GitHub, GitLab, or Bitbucket.

Git Workflow

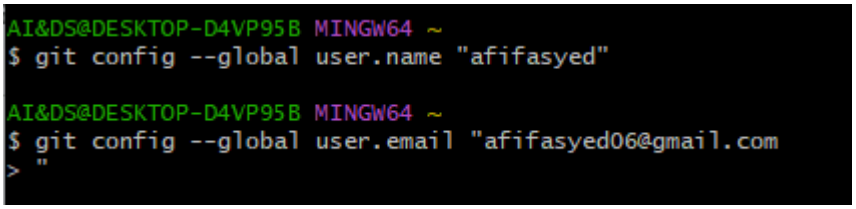
The basic Git workflow consists of four stages:

1. **Working Directory**
   - The place where project files are created or modified.
2. **Staging Area**
   - Files are prepared before committing using git add.
3. **Local Repository**
   - Changes are permanently saved using git commit.
4. **Remote Repository**
   - Changes are uploaded using git push.

Git Commands

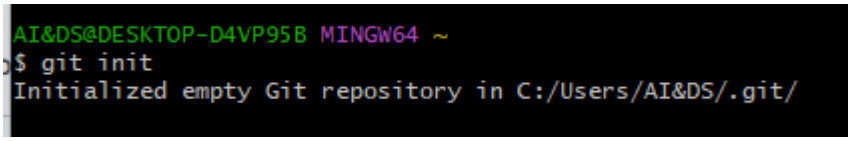
1. Configure Git

git config --global user.name "Your Name"
git config --global user.email "your@email.com"



2. Initialize Repository

Creates a new Git repository.



3. Check Repository Status

Displays modified and untracked files.

git status

```
AI&DS@DESKTOP-D4VP95B MINGW64 ~ (master)
$ git status
warning: could not open directory 'Application Data/': Permission denied
warning: could not open directory 'Cookies/': Permission denied
warning: could not open directory 'Local Settings/': Permission denied
warning: could not open directory 'My Documents/': Permission denied
warning: could not open directory 'NetHood/': Permission denied
warning: could not open directory 'PrintHood/': Permission denied
warning: could not open directory 'Recent/': Permission denied
warning: could not open directory 'SendTo/': Permission denied
warning: could not open directory 'Start Menu/': Permission denied
warning: could not open directory 'Templates/': Permission denied
On branch master

No commits yet

Untracked files:
```

4. Add Files

Adds files to the staging area.

git add filename

```
AI&DS@DESKTOP-D4VP95B MINGW64 ~ (master)
$ git add "C:\Users\AI&DS\Downloads\eg.txt"
```

To add all files:

git add .

5. Commit Changes

Saves staged changes permanently.

git commit -m "Initial Commit"

```
AI&DS@DESKTOP-D4VP95B MINGW64 ~ (master)
$ git commit -m "Initial Commit"
[master (root-commit) 322f127] Initial Commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Downloads/eg.txt
```

6. View Commit History

git log

```
AI&DS@DESKTOP-D4VP95B MINGW64 ~ (master)
$ git log
commit 322f1277ca207e6e7f8a2dad0b788f735ee1b0e4 (HEAD -> master)
Author: afifasyed <afifasyed06@gmail.com>
Date:   Thu Jul 23 11:33:57 2026 +0530

    Initial Commit

AI&DS@DESKTOP-D4VP95B MINGW64 ~ (master)
```

7. Create a Branch

git branch feature

```
AI&DS@DESKTOP-D4VP95B MINGW64 ~ (master)
$ git branch feature
```

8. Switch Branch

git checkout feature

or

git switch feature

```
AI&DS@DESKTOP-D4VP95B MINGW64 ~ (master)
$ git checkout feature
Switched to branch 'feature'
```

9. Create and Switch Branch

git checkout -b feature

or

git switch -c feature

```
AI&DS@DESKTOP-D4VP95B MINGW64 ~ (feature)
$ git switch -c feature
fatal: a branch named 'feature' already exists

AI&DS@DESKTOP-D4VP95B MINGW64 ~ (feature)
$ git checkout -b feature
fatal: a branch named 'feature' already exists
```

10. Merge Branch

git checkout main
git merge feature

```
AI&DS@DESKTOP-D4VP95B MINGW64 ~ (feature)
$ git merge feature
Already up to date.
```

11. Delete Branch

git branch -d feature

```
AI&DS@DESKTOP-D4VP95B MINGW64 ~ (feature)
$ git checkout master
Switched to branch 'master'

AI&DS@DESKTOP-D4VP95B MINGW64 ~ (master)
$ git branch -d feature
Deleted branch feature (was 322f127).
```

12. Connect to Remote Repository

git remote add origin https://github.com/username/repository.git

13. Push Changes

git push -u origin main

```
AI&DS@DESKTOP-D4VP95B MINGW64 ~ (master)
$ git remote add origin https://github.com/afifasyed123/repository.git

AI&DS@DESKTOP-D4VP95B MINGW64 ~ (master)
```

14. Pull Latest Changes

git pull origin main

```
AI&DS@DESKTOP-D4VP95B MINGW64 ~ (master)
$ git pull origin main
```

15. Clone Existing Repository

git clone https://github.com/username/repository.git

```
AI&DS@DESKTOP-D4VP95B MINGW64 ~ (master)
$ git clone https://github.com/afifasyed123/Task-Management-System
Cloning into 'Task-Management-System'...
remote: Enumerating objects: 49, done.
remote: Counting objects: 100% (49/49), done.
remote: Compressing objects: 100% (35/35), done.
remote: Total 49 (delta 2), reused 45 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (49/49), 318.19 KiB | 28.93 MiB/s, done.
Resolving deltas: 100% (2/2), done.

AI&DS@DESKTOP-D4VP95B MINGW64 ~ (master)
```

Branching

A **branch** is an independent line of development. It allows developers to work on new features or bug fixes without affecting the main project.

Benefits

- Parallel development.
- Easier bug fixing.
- Safe experimentation.

- Simplifies team collaboration.

Merging

Merging combines changes from one branch into another. It is commonly used after completing feature development.

Example:

git checkout main
git merge feature

Authentication (Authorization)

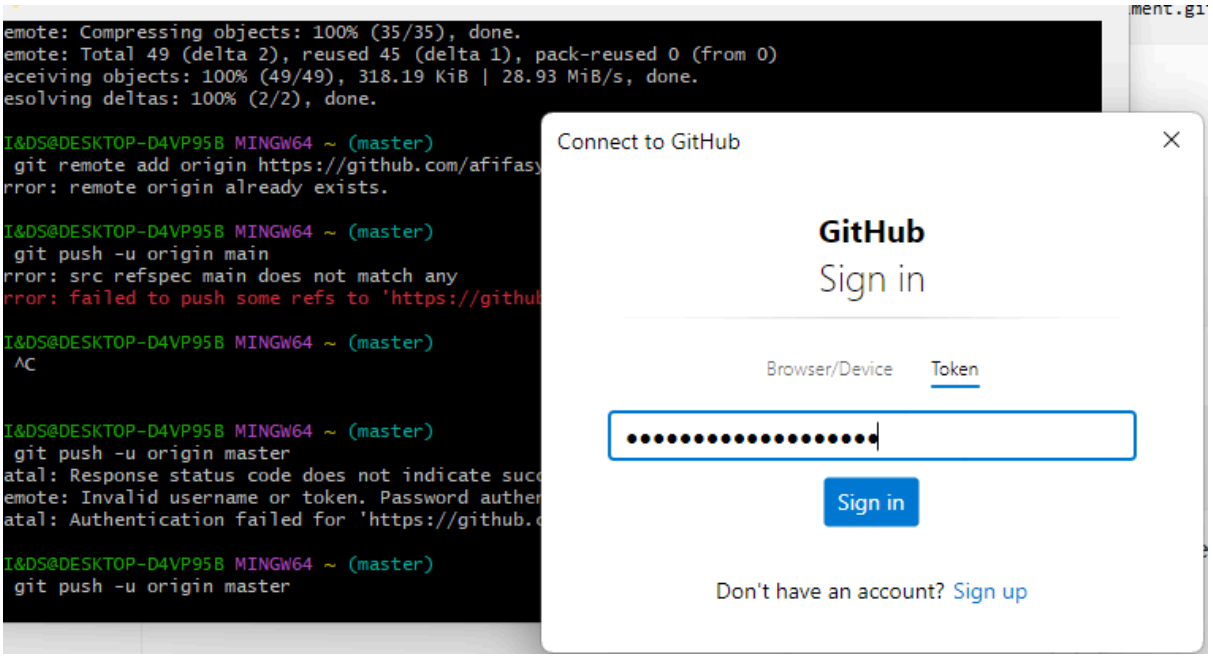
When pushing code to a remote repository, Git requires authentication.

Using Personal Access Token (HTTPS)

git push origin main

Enter:

- Username: GitHub username
- Password: Personal Access Token (PAT)



Using SSH

Generate SSH key:

ssh-keygen -t ed25519 -C "your@email.com"

Test authentication:

ssh -T git@github.com

Conclusion

The experiment was successfully completed by performing the basic version control operations using Git. A Git repository was created, files were added to the staging area, changes were committed, branches were created and merged, and the project was synchronized with a remote repository. This experiment demonstrated how Git helps manage source code efficiently, maintain version history, support collaboration, and ensure safe and organized software development.